Lab 2.b — Guardrails, and Making Standards Travel

Skill: turning a written rule into something that refuses to be violated. **Duration:** ~2 hours, facilitated. **Surface:** your own copy of this repo.

The challenge. Pick one of this repo's written rules and give it teeth: a hook that stops (or nudges) the agent while it writes, and a CI gate that stops anyone at merge time — both verified in both directions.

Why this block exists

You now have documents. Nothing enforces them.

The industry has converged on an uncomfortable finding: **the model layer cannot be the control.** Models are non-deterministic and persuadable. A rule in CLAUDE.md is a strong suggestion — it will hold most of the time and fail exactly when someone asks for something reasonable-sounding that happens to violate it. If a rule matters, something deterministic has to check it.

Three words for the rest of the block:

- A **guardrail** is any deterministic mechanism — a permission rule, a hook, a CI check — that refuses a violation instead of merely describing one.
- A **hook** is an interception point — an event fired right before or after a consequential action.
- A **policy** is the deterministic rule evaluated against that event.

Because the check doesn't depend on which model ran, it still holds when you swap models or add agents.

Before the session

- [ ] Your copy runs and `npm test` is green
- [ ] You've done the Claude Code **Hooks** lesson (Extending Claude Code) *(not yet? Do it before the session — the demo assumes it. The 15-minute core is enough)*
- [ ] Pull the snippets folder if you haven't:

```
npx giget gh:mike-tajmajer-fullstacklabs/fsl-taskboard-lab/docs/labs docs/labs
```

The five layers

Layer	Where it lives	Holds against
1. **Instruction**	CLAUDE.md, docs behind a trigger table	Nothing, reliably. It informs; it does not enforce
2. **Claude hooks + permissions**	`.claude/settings.json` — deterministic, client-side	The agent, at authoring time
3. **Commit-time**	git hooks, commit lint	Anyone committing locally
4. **CI gates**	workflows, static analysis, custom validators	Anyone merging, agent or human
5. **Documented bypass**	a written escape hatch with a named approver	Nothing — it's what keeps the other four honest

This repo ships guardrails only at layers 1–2 — its CI runs lint and tests, but carries no *governance* gate yet. That gap is deliberate, and it's yours: today you build in layers 1–2 and wire **one layer-4 gate** so you feel what it costs. Layer 3 you'll watch get built live — adding it to *your copy* is a stretch goal with every step written down. Layer 5 is taught, and you document one.

The session tells the enforcing layers as a **timeline** — the same anatomy (an event fires a script; the exit code decides) at three moments: **authoring** (a Claude hook stops the agent), **commit** (a git hook stops anyone committing), **merge** (a CI gate stops anyone, full stop).

(If you meet "Tier" language elsewhere in this repo's docs, note it counts the other way: Tier 1 there means the server-side backstop — this handout's layer 4.)

Warm-up: the cheapest guardrail there is (~5 min, everyone)

`.claude/settings.json` takes three verdicts, not two: **allow**, **ask**, **deny**. No code.

Here is what your file looks like **before** — this is what must survive the merge:

```
{
  "permissions": {
    "deny": ["Edit(server/data/seed.json)", "Write(server/data/seed.json)"]
  },
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "node \"$CLAUDE_PROJECT_DIR/.claude/hooks/protect-seed.js\""
          }
        ]
      }
    ]
  }
}
```

(A `settings.local.json` may sit next to it — that's your personal, git-ignored overrides file; leave it alone today.)

> **Merge these in — don't paste over the file.** Your `.claude/settings.json` already has a `permissions.deny` block holding the two `server/data/seed.json` rules, and a `hooks` block registering `protect-seed.js`. Both must survive: they're layers 1–2 of the exhibit you'll read in the next section, and the demo depends on them. Add the `ask` array, and **append** to the existing `deny` array.

```
{
  "permissions": {
    "allow": [],
    "ask": ["Bash(rm *)"],
    "deny": [
      "Edit(server/data/seed.json)",
      "Write(server/data/seed.json)",
      "Bash(rm -rf *)",
      "Bash(git push --force *)",
      "Bash(git reset --hard *)",
      "Bash(git clean *)",
      "Bash(npm publish *)",
      "Bash(curl * | bash)"
    ]
  }
}
```

The first two `deny` entries are the ones already in your file — shown so you can see where yours go, not so you add them twice. The `allow` array is empty **on purpose**: it's the third verdict's home, where you later promote commands you never want to be asked about (`Bash(npm test)`); entries are earned, and empty is the right day-one default. Once merged, ask Claude to do one of these things and watch it refuse. Then confirm you didn't break anything: ask it to edit `server/data/seed.json` and check that it is still refused.

This is the honest answer to "there are no restore points" and "it did something I didn't expect." It takes five minutes, needs no code, and works in every repo you own — including the ones you're shipping from tomorrow. If you take one thing from this block, take this. *(Written down step-by-step in `lab-2b-demo-1-permissions-walkthrough.md` .)*

The hook you already know — layer 3, seen live

Your instructor demos a **git hook** before any Claude hook, because your team already runs one (husky, on your own repo) — and because it teaches the anatomy everything else in this lab repeats: **an event fires a script, and the exit code decides.**

The demo enforces a rule this repo already states and never checks — `CLAUDE.md` says *"Conventional Commits"*:

```
npm install -D husky
npx husky init
rm .husky/pre-commit                                  # the starter hook (npm test) runs
        BEFORE commit-msg
cp docs/labs/snippets/commit-msg.sh .husky/commit-msg
git commit --allow-empty -m "changed stuff"           # → BLOCKED, commit never happens
git commit --allow-empty -m "docs: explain it"        # → passes, silently
git commit --no-verify -m "changed stuff"             # → the bypass that ships with git
```

(Every line is identical on Windows — `cp` is a PowerShell alias. And no `chmod` , no file extension: git runs the hook through husky's shim and its own bundled shell, so the file is never "executed" by your OS. Why that works — and the raw-git-hooks caveat where `chmod` IS real — is step 3 of the walkthrough.)

Two honest caveats, and both are the argument for layer 4: the hook is **per-clone** (a fresh clone without `npm install` has no hooks), and `--no-verify` skips it at will. Neither escape exists at merge time.

One anatomy, three moments:

Moment	Mechanism	Input	Verdict	Holds against
Authoring	Claude hook	tool input, JSON (stdin)	exit 2 → blocked; stderr → agent	the agent
Commit	git hook	commit-message file	non-zero exit → no commit	anyone committing, wired clone
Merge	CI gate	the diff vs base branch	failed check → no merge	anyone merging — agent or human

Doing this in your own copy is a **stretch goal** — every step, both verifications, and the cleanup are in `lab-2b-demo-2-git-hook-walkthrough.md`.

The three-layer pattern, already in this repo

`server/data/seed.json` is protected three times over, and every layer names the same path and the same remediation:

1. A **Don't** line in `CLAUDE.md`
2. A `permissions.deny` entry in `.claude/settings.json`
3. A `PreToolUse` hook, `.claude/hooks/protect-seed.js`

Read that hook before you write yours — particularly its message, which tells Claude what was blocked, **why**, what to do instead, and what done looks like. A guardrail that only says "no" makes the agent guess.

Block or nudge? Decide, don't default

The exit code you use, on the event you register, *is* the severity decision:

Event	Exit 2 does	Use for
`PreToolUse`	**Blocks** the call; stderr goes to Claude	Unrecoverable or unambiguous violations
`PostToolUse`	Cannot block — the tool ran — but stderr still goes to Claude	Heuristic rules; things you don't want to interrupt

Not every guardrail should block. A gate that fires on legitimate work teaches people to route around it, and a bypassed gate is worse than no gate because it looks like control.

What you'll do

1. Build along: the read-only-database guardrail

Your instructor extends the read-only-database rule — and **you build the same thing in your own copy as they go**: CLAUDE.md line → deny rule → hook → **verified in both directions** → bypass documented. The verify step lets you hear both layers answer — the deny's generic wall first (permission rules evaluate before hooks), then, with the deny briefly removed, your hook's four-part signpost; the difference between them is the lesson. The whole build is written down step-by-step with the code in `lab-2b-demo-3-claude-hook-walkthrough.md` — replay it after class, or use it to fix a step that didn't land live. It's the git-hook anatomy moved to authoring time: the event is the agent's tool call, the input arrives as JSON on stdin, and `exit 2` is the block.

Two clarifications worth having before you start. First, run `npm run reset-db` so `db.json` exists — it's generated and git-ignored, so a fresh copy doesn't have it. Second, this rule ("don't hand-edit `db.json`") is a *different rule* from the menu's store boundary ("only `server/src/repositories/` may *import* `db/store.ts`") — same file family, two rules, and today's hook enforces the first.

Checkpoint: in *your* copy, Claude is refused when it edits `server/data/db.json` and succeeds when it edits a file in `server/src/repositories/`.

2. Pick a rule and enforce it — the hook

The law is already written; you're adding the police. Pick **one** rule — or bring a rule of your own (the only filter: a script must be able to detect it from a file path or file content):

The rule	Where it is written	Hook shape
A server/src change ships with a sibling test	CLAUDE.md · testing approach	`PostToolUse` nudge — edited `*.ts` has no sibling `*.test.ts`
Only repositories/ may import db/store	CLAUDE.md · Don't list	`PreToolUse` block — written content imports `db/store`
Throw typed errors, not `new Error()`	CLAUDE.md · code conventions	`PostToolUse` nudge — content has `throw new Error(`
Responses only via respond.ts	ADR-0001 + CLAUDE.md	`PostToolUse` nudge — route content has `res.json` / `res.send`

No wrong choice — the hook-shape column is a hint, not a spec. Then start from `docs/labs/snippets/hook-skeleton.js`: copy it to `.claude/hooks/check-convention.js`, delete the two example rules, put yours in.

Then **register it** — the hook does nothing until `.claude/settings.json` names it:

```
"hooks": {
  "PostToolUse": [
    {
      "matcher": "Edit|Write",
      "hooks": [{ "type": "command", "command": "node
        \"$CLAUDE_PROJECT_DIR/.claude/hooks/check-convention.js\"" }]
    }
  ]
}
```

- `matcher` — which tool calls trigger it (here: any Edit or Write).
- `$CLAUDE_PROJECT_DIR` — resolves to the repo root, so the path works on every machine.
- Pick the **event** for the severity you chose: `PreToolUse` blocks, `PostToolUse` nudges.
- **Start a new session** after registering — hooks load at session start.

One decision, made by the event you register on: **path rules can block; content heuristics should nudge** — an imperfect block stops real work, an imperfect nudge costs one ignorable note.

Verify both directions, concretely:

1. **Violation case:** ask Claude to break your rule on purpose. Save the hook's text.
2. **Allow case:** ask for a legitimate edit the rule shouldn't touch — the hook must stay silent.
3. **Evidence:** paste the hook's text into your PR description. That's what "demonstrated in a live agent run" means.

Checkpoint: your hook fires on the violation, stays silent on a legitimate edit, and its text is in your PR.

3. The same rule at merge time — the CI gate

The hook stops the agent. This stops anyone — it's the moment `--no-verify` can't reach. Your gate joins the checks that already run on every PR (lint · typecheck · tests — read them in `lab-2b-demo-4-ci-walkthrough.md`, which also hides a find-the-unenforced-rule exercise). Start from `docs/labs/snippets/governance-gate.yml`, copy it to `.github/workflows/governance.yml`, and encode **your rule's check** — the same detection your hook runs, applied to the PR's changed files. Note the `fetch-depth: 0` comment — without it any diff against the base branch fails with an unhelpful error. And the gotcha that catches half of every cohort: **the gate runs on `pull_request`** — push your branch and **open a draft PR**, or nothing runs and your gate will look broken when it's merely unasked.

Checkpoint: your gate fails a PR that violates your rule and passes one that doesn't.

(Notice your gate's behavior on a docs-only PR — if it fires there, a path filter is the one-line fix.)

How it's assessed

Lab rubric + peer review: **Completion · Quality · Process · Independence · Insight**, 1–4 each. The guardrail itself is pass/fail — it must demonstrably fire *and* stay silent. Your verification story carries the Insight score. This lab completes the **Agentic Developer** requirements at the week-4 gate, and your hook or CI gate also qualifies as the custom agentic artifact for **Framework Practitioner** at the end of Phase 2.

Acceptance criteria

- [ ] The **warm-up** allow/ask/deny list is merged into your `.claude/settings.json` — seed rules and hook intact — and you've seen it refuse something
- [ ] A **hook** in `.claude/hooks/`, registered, enforcing a written rule of this repo (from the menu, or one of your own)
- [ ] A **CI gate** in `.github/workflows/governance.yml` running the same rule's check
- [ ] **Both verified in both directions** — the hook's text and the gate's red-then-green run in your PR
- [ ] Demonstrated **firing in a live agent run**, PR opened, CI green
- [ ] A **reflection note** — 3–5 sentences in your PR description: what you learned, what surprised you, what you'd apply at work tomorrow

Stretch

- Wire **layer 3 in your copy**: husky + the commit-msg hook, verified in both directions — every step is in `lab-2b-demo-2-git-hook-walkthrough.md`
- Compose several rules into one policy bundle
- Add telemetry: count how often the gate fires and how often it's bypassed. A gate with no telemetry can't be falsified
- Extend the existing lint setup to complete a half-enforced rule
- Take the takeaway card and sort one of your **own** conventions into mechanical and fuzzy clauses

Common stuck points

Symptom	What to do
The hook fires when it shouldn't	Narrow the trigger — then **re-verify the allow case**. Fixing a false positive by loosening the block is how gates die
The hook never fires	Check the event and the `matcher`. Claude Code logs which hooks matched and their exit codes — run with `--debug`
"My detection regex is imperfect"	That's what nudges are for — an imperfect block stops real work; an imperfect nudge costs one ignorable note
Tempted to block everything	Ask what happens the first time it fires on legitimate work. You'll be the reason someone disables hooks
The CI gate fails on a docs-only PR	Expected. Now decide: path filter, warn instead of fail, or accept it. That decision is the deliverable
`git diff` fails in CI with "unknown revision"	`fetch-depth: 0` on the checkout step
The git hook never fires (stretch)	`npm install` run since husky landed? `"prepare": "husky"` present? `git config core.hooksPath` says `.husky/_`?
The git hook blocks a message you meant	Read its message — the regex *is* the policy. Extend the type list on purpose, in a PR, not by reaching for `--no-verify`
Every commit: `npm error Missing script: "test"`	Husky's starter `pre-commit`, not your hook — it runs before `commit-msg`. Delete it, or add a test script
Every commit blocked: `: command not found` (127)	The hook file has Windows CRLF line endings — save it as LF (VS Code: status-bar CRLF → LF)

The artifact that's yours to define

After today you have a governance artifact in one repo — and more repos ready for it.

How a standard reaches every repo, and how anyone knows they're current, is itself a governance artifact — and it's the one we deliberately leave in your hands. It sits at the **Collective** layer, above any single project — the **sixth artifact**, next to the five that live inside a repo (CLAUDE.md, ADR, NFR, recipe, rules file).

Why it's yours: choosing the mechanism, the starting rung, and the owner *is* the roll-up to a Collective Brain. A team that receives its distribution model has been handed a standard rather than adopting one.

Notice you already used the first two rungs today. **This handout** reached you as one canonical repo plus a pointer — that's *crawl* — and if you ran the `giget` command, that's *walk*. Neither took a decision from anyone in this room; the third rung will.

`distributing-standards.md` has the options in full — crawl (~30 min), walk (minutes to make it queryable, 1–2 hrs to pin copies), run (a half-day to two days, plus an owner) — with an honest trade-off table and a template for recording what you choose.

What "done" looks like: a recorded decision naming the **rung**, the **first move**, and an **owner who is a person, not a committee.** Crawl is about thirty minutes of one person's afternoon, so "we'll decide later" is the only answer that costs more than acting. A rung with no owner is a rung nobody is on.

Where this goes next

Three things leave this room and continue outside it:

- **Port one artifact to a real repo.** Pick the smallest thing that transfers — usually the deny/ask list, sometimes an NFR — and land it in a repo you actually ship from. Office hours is for this; bring the repo, not a question. This is the step that decides whether Lab 2 produced a standard or a training exercise.
- **Define how standards travel** — the artifact above. Owner named, decision recorded, tracked in the weekly status. Office hours and the champions track support it.
- **Apply the pattern to a convention that's yours.** `enforcing-a-convention.md` is the takeaway card — it works the whole shape through on stored procedures, including the false positives and the clauses no gate can decide. Independent follow-up, supported in office hours.